

试题1、线性电路直流（DC）仿真

1. 背景

直流仿真是用于确定电路静态工作点的基本方法。我们可以对电路施加恒定的电压或电流源，并求解由此产生的代数方程，以找到电路中各点的直流电压和电流值。通过直流仿真，可以了解电路在静态条件下的行为，这对后续的交流分析和瞬态仿真非常重要。

对于线性电路，使用改进的节点分析（Modified Nodal Analysis, MNA）方法，我们可以直接生成线性代数方程组，并通过矩阵方程求解的数值方法（如高斯消去法、LU 分解法、迭代法等）求解这些方程。MNA 方法基于 KCL 定律构建方程。以图 1 为例，该图给出了一个仅仅包含 3 个电阻和一个电流源的简单电路示意图，各元器件参数已在图中标注。红色圆圈标注了 3 个节点的编号。我们可以针对节点 1 和节点 2，建立如下两个 KCL 方程：

$$\frac{v_1}{R_1} + \frac{v_1 - v_2}{R_2} = I_1, \quad (1)$$

$$\frac{v_2 - v_1}{R_2} + \frac{v_2}{R_3} = 0. \quad (2)$$

其中， v_1 和 v_2 分别表示节点 1 和节点 2 的节点电压。以上两个方程中共有 2 个变量，方程数目与未知变量数目相同，可以直接求出 v_1 和 v_2 。方程(1)和(2)可以重写为矩阵方程的形式：

$$\begin{bmatrix} \frac{1}{R_1} + \frac{1}{R_2} & -\frac{1}{R_2} \\ -\frac{1}{R_2} & \frac{1}{R_2} + \frac{1}{R_3} \end{bmatrix} \cdot \begin{bmatrix} v_1 \\ v_2 \end{bmatrix} = \begin{bmatrix} I_1 \\ 0 \end{bmatrix}. \quad (3)$$

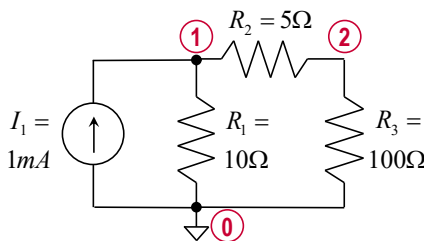


图 1 用于建立电路方程的简单电路示意图

一般而言，线性电路用于 DC 分析的 MNA 方程都可以写成类似(3)的矩阵方程形式：

$$\mathbf{A} \cdot \mathbf{x} = \mathbf{b}。 \quad (4)$$

其中 $\mathbf{A}$ 表示导纳矩阵， $\mathbf{v}$ 表示节点电压向量， $\mathbf{b}$ 表示外接电流源激励。上述公式中每一行代表一个线性方程。等号左边表示从各节点流出的电流，等号右边表示流入各节点的电流。

2. 常用线性方程组数值求解算法

常用的用于求解矩阵线性方程的数值方法包括高斯消去法（Gaussian Elimination）、LU 分解（LU Decomposition）、雅可比迭代法（Jacobi Method）、高斯-赛德尔迭代法（Gauss-Seidel Method）等。

(1)、高斯消去法

高斯消去是最基本的直接求解方法，通过对矩阵 $\mathbf{A}$ 进行一系列的行变换，将矩阵化为上三角形式，然后进行回代求解。假定需要求解的 MNA 方程如下：

$$\begin{bmatrix} 2 & 1 & -1 \\ -3 & -1 & 2 \\ -2 & 1 & 2 \end{bmatrix} \cdot \begin{bmatrix} v_1 \\ v_2 \\ v_3 \end{bmatrix} = \begin{bmatrix} 8 \\ -11 \\ -3 \end{bmatrix}。 \quad (5)$$

采用高斯消去法，我们首先利用行变换，利用(5)的第一行消去其第二行和第三行的第一个元素（这时，公式(5)第一行第一个元素被称为主元（pivot））：

$$\begin{bmatrix} 2 & 1 & -1 \\ 0 & 0.5 & 0.5 \\ 0 & 2 & 1 \end{bmatrix} \cdot \begin{bmatrix} v_1 \\ v_2 \\ v_3 \end{bmatrix} = \begin{bmatrix} 8 \\ 1 \\ 5 \end{bmatrix}。 \quad (6)$$

然后利用(6)的第二行消去其第三行第二个元素（这时，公式(6)第二行第二个元素被称为主元），得到上三角矩阵：

$$\begin{bmatrix} 2 & 1 & -1 \\ 0 & 0.5 & 0.5 \\ 0 & 0 & -1 \end{bmatrix} \cdot \begin{bmatrix} v_1 \\ v_2 \\ v_3 \end{bmatrix} = \begin{bmatrix} 8 \\ 1 \\ 1 \end{bmatrix}。 \quad (7)$$

再进行回代，从最后一行开始依次求出 v_3 、 v_2 和 v_1 ：

$$v_3 = -1, \quad (8)$$

$$0.5 \cdot v_2 - 0.5 \cdot v_3 = 1 \Rightarrow v_2 = 3, \quad (9)$$

$$2 \cdot v_1 + 3 \cdot v_2 + 1 \cdot v_3 = 8 \Rightarrow v_1 = 2. \quad (10)$$

高斯消去法尽管是一种有效的线性方程组求解方法，但在实际应用中也存在一些缺陷和局限性。例如，在计算过程中，特别是涉及浮点数运算时，舍入误差可能会累积，从而影响最终结果的精度。此外，如果矩阵中存在非常小的主元，可能会导致数值不稳定，进一步加剧舍入误差。这种情况下，需要进行主元选取（**Partial Pivoting**）或完全主元选取（**Full Pivoting**）来改善稳定性。另外，高斯消去法在处理稀疏矩阵时效率较低，因为它通常会填充零元素，导致生成较多的非零元素，使得矩阵变得稠密，增加了存储和计算的开销。对于病态矩阵（即条件数非常大的矩阵）、矩阵方程无解或有无穷多个解的情况，直接使用该算法也存在很多问题。

(2)、 LU 分解法

有时候我们希望改变给定电路的输入，即对同一个电路在不同激励下进行直流分析（即公式(4)等号右边的向量 **b** 发生了变化）。这时直接采用高斯消去，会需要多次进行行变换来构造多个上三角矩阵，复杂度较高。利用 LU 分解法，则可以避免这一过程。采用该方法，我们可以将矩阵 **A** 分解为一个下三角矩阵 **L** 和一个上三角矩阵 **U**：

$$\mathbf{A} = \mathbf{L} \cdot \mathbf{U}. \quad (11)$$

一般我们假设 **L** 矩阵对角线上的元素全部为1。然后通过求解前向替换求解

$$\mathbf{L} \cdot \mathbf{y} = \mathbf{b} \quad (12)$$

得到 **y**。再通过向后替换求解：

$$\mathbf{U} \cdot \mathbf{x} = \mathbf{y} \quad (13)$$

就可以得到 **x**。即通过求两个三角方程组来得到解。若输入 **b** 发生变化，我们只需要重新求解两个三角矩阵方程(12)和(13)即可得到节点电压 **x**，而不需要重新对 **A** 进行 LU 分解。与采用高斯消去法相比，计算复杂度要降低很多。

(3)、 Jacobi 迭代法

Jacobi 迭代法是一种用于求解线性方程组的迭代方法，尤其适用于大型稀疏矩阵。其基本思想是通过逐次近似逐步逼近精确解。对于线性方程组(4)，我们可以将矩阵 $\mathbf{A}$ 分解为对角矩阵 $\mathbf{D}$ 、严格下三角矩阵 $\mathbf{L}$ 和严格上三角矩阵 $\mathbf{U}$ 的和：

$$\mathbf{A} = \mathbf{D} + \mathbf{L} + \mathbf{U}, \quad (14)$$

其中 $\mathbf{D}$ 只包含 $\mathbf{A}$ 的对角元素， $\mathbf{L}$ 包含 $\mathbf{A}$ 的下三角部分（不包括对角线）， $\mathbf{U}$ 包含 $\mathbf{A}$ 的上三角部分（不包括对角线）。然后将原方程组(4)重新写成：

$$\mathbf{D} \cdot \mathbf{x} = \mathbf{b} - (\mathbf{L} + \mathbf{U}) \cdot \mathbf{x}. \quad (15)$$

根据迭代法的基本思路，我们可以通过逐次近似来逼近解：

$$\mathbf{x}^{(k+1)} = \mathbf{D}^{-1} \cdot [\mathbf{b} - (\mathbf{L} + \mathbf{U}) \cdot \mathbf{x}^{(k)}]. \quad (16)$$

其中 $\mathbf{x}^{(k)}$ 表示第 k 次迭代得到的解， $\mathbf{x}^{(k+1)}$ 表示第 $k + 1$ 次迭代得到的解。一旦给定初值 $\mathbf{x}^{(0)}$ ，我们就可以根据迭代表达式(16)求出线性方程组(4)的数值解了。

Jacobi 迭代法提供了一种简便的方式来求解大型稀疏矩阵的线性方程组，特别适合并行计算和处理规模较大的问题。但是，相对于其他迭代方法（如 Gauss-Seidel 方法），Jacobi 迭代法的收敛速度较慢，尤其是对于某些矩阵，可能需要大量迭代才能达到满意的精度；而且数值稳定性较差，容易受到舍入误差的影响；内存占用上的成本也较高。

3. 试题要求

基于所提供的 benchmark 中用于线性电路 DC 分析的 MNA 方程，选择适当的数值求解算法，采用 matlab 脚本编写程序，求解节点电压 $\mathbf{x}$ 。建议：

- 实现高斯消去法时，可以考虑主元交换/主元消元法（即选择合适的主元）等来提高算法的稳定性。
- 对于有多种输入（即 $\mathbf{A}$ 矩阵保持不变，但是 $\mathbf{b}$ 向量发生变化）的 benchmark，请采用 LU 分解法进行求解。
- 可考虑并行等其他提高计算效率或者降低内存成本的方案。
- 提交 matlab 脚本及报告，报告中应给出每个 benchmark 的求解精度（建议采用 L2 范数 $\|\mathbf{Ax}_s - \mathbf{b}\|_2$ 来表示求解精度，其中 $\mathbf{x}_s$ 表示求出的数值解）、运行时间和内存占用情况。

4. Benchmarks

本题供需完成 3 个 benchmark 的求解。以下每一个.mat 文件对应 1 个 benchmark，都包含一个矩阵 $\mathbf{A}$ ，一个（或多个）向量 $\mathbf{b}$ （或者 $\{\mathbf{b}_1, \mathbf{b}_2, \mathbf{b}_3, \dots, \mathbf{b}_n\}$ ），采用 matlab 软件 load 各个.mat 文件即可看到。

- MNA_1.mat
- MNA_2.mat
- MNA_3.mat

试题2、自动布线算法

1. 背景

VLSI 自动布线问题是 VLSI 设计中的一个关键问题，其目的是在规定的版图区域内实现电路各个模块之间的物理连接，以确保在实体物理电路中实现正确的信号传输路径。自动布线通常基于版图网格实现。版图网格将整个设计区域划分为一系列等间距的网格点，使布线过程可以在一个确定的框架内进行。版图网格中，每个单元格大小相同，边长一般是由最小线宽和最小线间距决定。网格中布线冲突可以通过简单的单元格状态（即是否被阻塞）检查来检测 and 解决，以避免不同线网占用同一个单元格。网格平面上的走线要求严格水平或者垂直，这样能够有效管理走线资源。

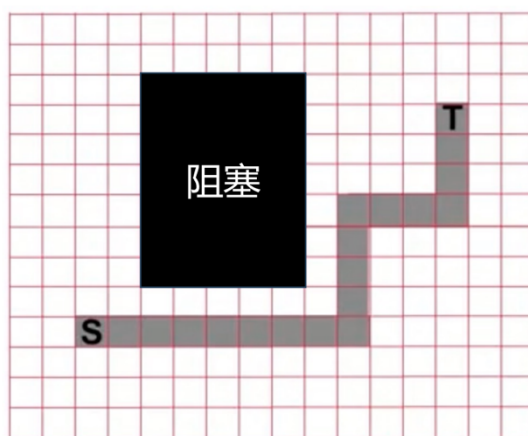


图2 包含阻塞单元的双端口线网布线示意图

图 2 展示了一个包含阻塞单元的双端口线网布线示意图，其中 S 和 T 分别表示线网的两个端点。目标是在起点 S 和终点 T 之间找到一条合法路径。为了简化自动布线问题，规定如下：

- **端点位置：**线网的所有端点都位于单元格的中心。
- **合法路径：**合法路径由一组连续相连的单元格构成。
- **阻塞区：**路径不能经过已经被阻塞的单元格（或已经存在布线的单元格）。
- **移动方向：**路径只能严格水平或垂直移动，即只能向东、南、西、北四个方向移动，不能斜线移动。
- **最短路径：**为了保证信号完整性及信号延迟，同时降低功耗和制造成本，通常布线时我们要尽量保证布线路径最短。

此外，在现代 VLSI 设计中，还有一个典型的布线应用场景是多通道总线布线。为了在电路单元之间并行地传输信号，需要布置由多根信号线构成的总线。在某些情况下，总线中的各个比特位信号需要同步传输。实现这一要求的一种方式是在控制总线中各个信号线之间的长度偏差在一定范围内，甚至完全相等。图 2 展示了一个三通道总线的布线实例。该总线布线连接了三对端口： $(P_{1,1}, P_{1,2})$ 、 $(P_{2,1}, P_{2,2})$ 、 $(P_{3,1}, P_{3,2})$ ，并绕过了阻塞 O_1 和 O_2 。为了实现信号同步传输，这些布线的长度相等。

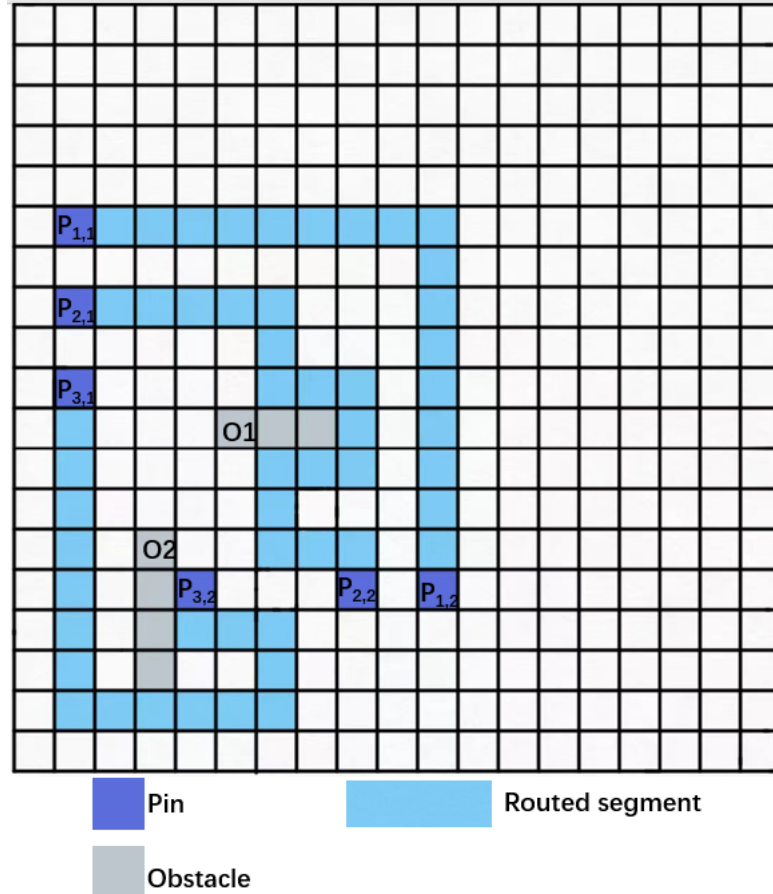


图 3 三通道总线布线示例。

2. 迷宫布线算法

迷宫算法 (Maze Routing Algorithm) 是一种用于 VLSI 自动布线的经典算法。它的基本思想是采用广度优先搜索 (Breadth-first search, BFS), 通过系统地探索可能的路径来找到连接两点的最短路径。基本的迷宫布线目标是找到从起点到终点的最短路径 (最少步数), 可直接用于双端线网的自动布线。

图 4 给出了利用迷宫算法完成单个双端线网布线示例。由此可见基本的迷宫算法主要步骤如下:

- **初始化:** 在网格上标记起点和终点。然后将起点加入一个队列, 表示开始搜索的位置。
- **波前扩展:** 从队列中取出当前节点, 标记该节点已被访问。检查当前节点的四个相邻节点 (上、下、左、右)。对于每个相邻节点, 如果该节点是空的且未被访问, 则将

其加入队列，并记录从起点到该节点的步数。具体而言，如图 4 所示，我们从源点 S 出发，寻找所有经过一个步长能到达的单元格，标记为 1（路径长度为 1）。利用标记为 1 的单元，寻找所有经过两个步长能到达的单元，标记为 2（路径长度为 2）。重复这一过程，继续标记直到到达终点。

- **路径回溯：**当终点被访问时，表示找到了从起点到终点的路径。从终点开始回溯，通过记录的步数，依次找到到达终点的路径上的每个节点，直到回到起点。如图 4 所示，我们找到了蓝色和绿色两条路径
- **路径标记：**将找到的路径标记为最终的布线路径。

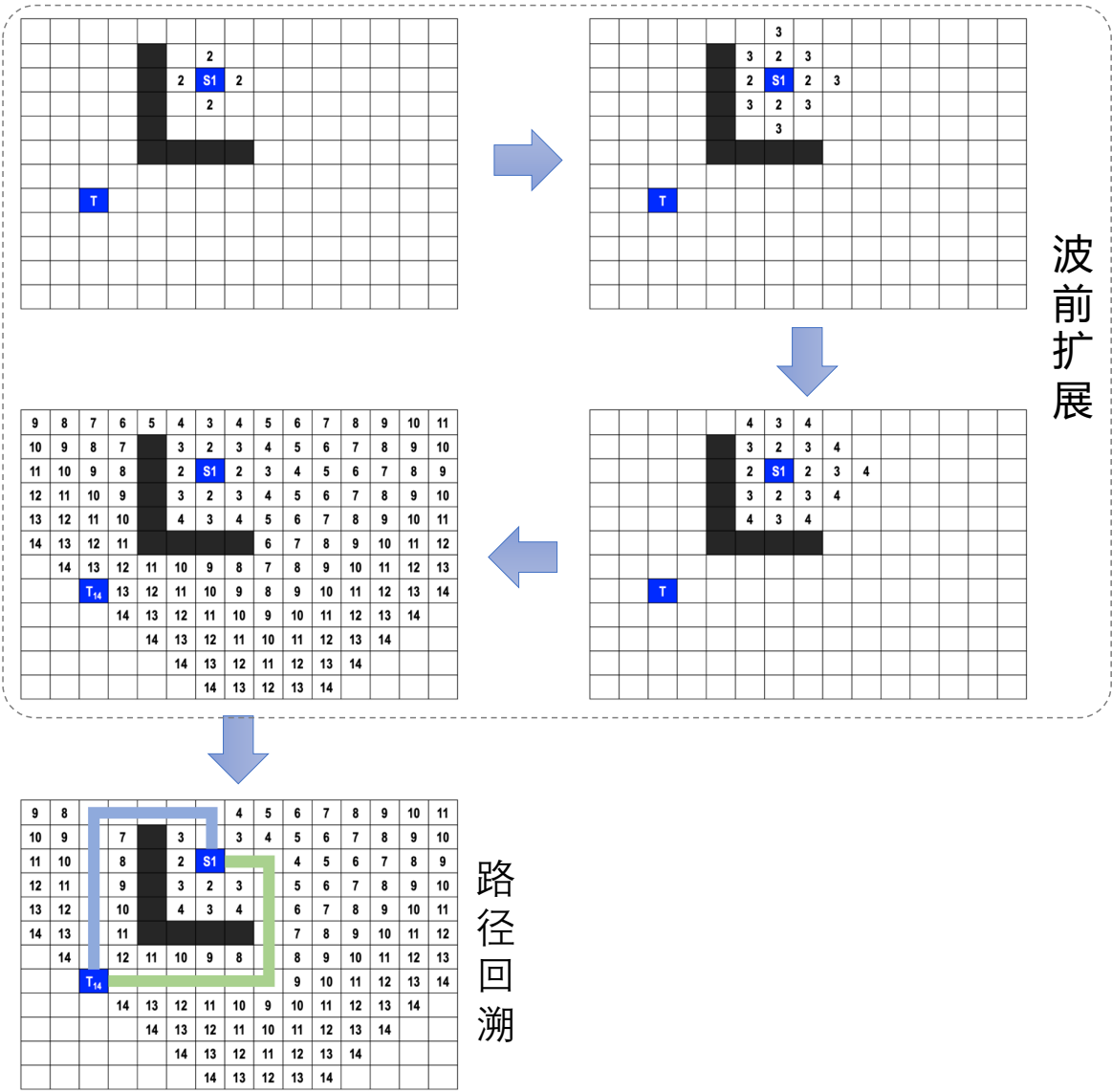


图 4 利用迷宫算法完成双端线网布线示例

值得注意的是，对于多个线网，我们可以一次仅完成一个线网的布线。然后将得到的该线网从 S 到 T 的路径上的单元格标记为阻塞（block），后续线网的布线不能使用这些单元格。完成后再处理下一个线网。在对下一个线网的布线时，网格中以前布线用到的单元都成为阻塞，只能利用空白的单元格进行布线。因此，早期线网的布线结果可能会使得后期线网无法得到最优布线，甚至导致后期线网因阻塞无法布线。

此外，迷宫算法还可拓展用于多端口线网布线。如图 5 所示，首先采用任选一个端口做为起点，其他端口为终点，开始波前扩展，直到找到第一个终点，并进行回溯，得到起点到该终点的路径。然后将该路径上的所有单元格标记为起点，重新开始进行波前扩展，直到找到第二个终点，并进行回溯，得到第二条路径。值得注意的是，第二条路径的起点可能是第一条路径上的任意单元格。

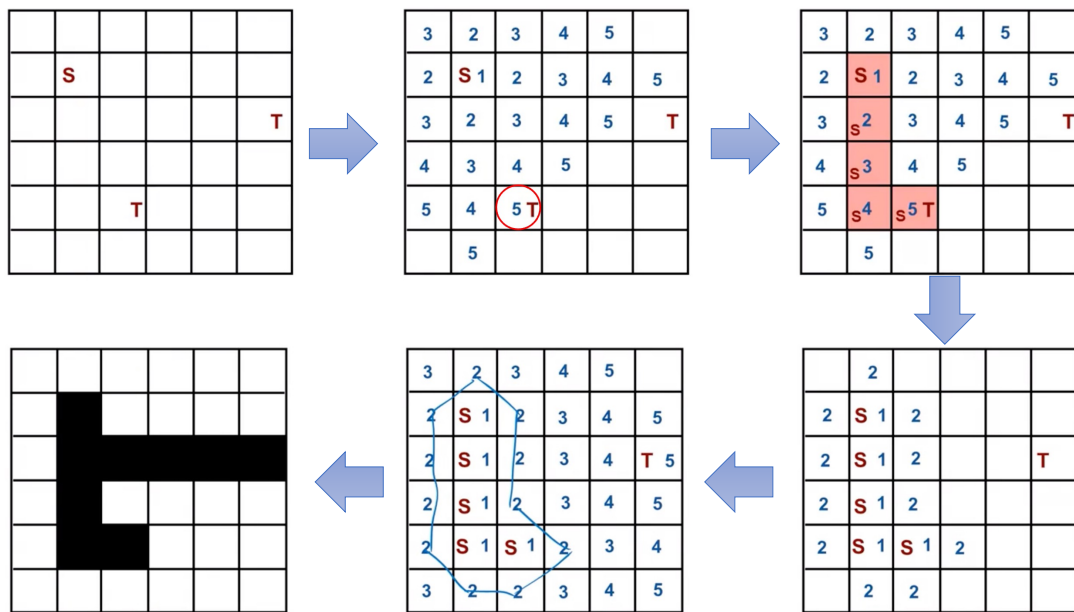


图 5 用于三端口线网布线的迷宫算法示意图

3. 试题要求

采用 matlab 实现基于迷宫算法的自动布线算法，包括：

- 实现基于迷宫算法的单个多端口线网布线方法。
- 实现基于迷宫算法的多个多端口线网布线方法，探索如何确定最佳布线顺序，即到底哪些线网应该先布，哪些线网应该后布。

-
- 实现总线布线算法，保证总线中的所有线网的长度完全相等。并在保证长度相等的前提下，路径越短越好。
 - 探索启发式算法（如 A* 算法等）以提高算法效率。
 - 提交 matlab 脚本及报告，报告中应给出每个 benchmark 各个线网的路径长度、所有线网总长度、布线结果示意图（采用画图函数 `display2DGrid(grid, nets)` 得到）、算法运行时间和内存占用量。

4. Benchmarks

以下每一个 .mat 文件中都包含 4 个变量：`row_num`（单层平面网格的行数）、`col_num`（单层平面网格的列数）、`net_num`（线网的总数）和 `nets`（元胞数组：包含所有需要完成布线的线网）。元胞数组 `nets` 中的每个单元都代表一个线网，用一个 $n \times 2$ 列的矩阵表示，其中 n 表示该线网的端口数量。矩阵中的第一列表示对应线网中每个端口在平面网格中的行号，第二列表示每个端口在平面网格中的列号。注意，平面网格中第一行的行号为 1，其他行的行号向下依次增加；第一列的列号为 1，其他列的列号向右依次增加。例如图 5 中 3 个端口（1 个 S 和 2 个 T）的线网，对应的矩阵为：`[2 2; 5 3; 3 6]`。

- `testbench1.mat`: 50*50 的单层平面网格，共 1 个线网（4 个端口）需要布线；
- `testbench2.mat`: 100*100 的单层平面网格，共 5 个相互独立的线网（包含 3 个双端口线网，1 个三端口线网和 1 个四端口线网）需要布线，注意线网的布线顺序；
- `testbench3.mat`: 150*150 的单层平面网格，有 2 个相互独立的普通线网（分别有 3 个和 4 个端口）和一组总线（包含有 4 根双端口线网，要求这 4 个线网的布线长度相同即可）。

此外，为了增强布线结果的可视性，我们提供了画图函数 `display2DGrid(grid, nets)`（保存在文件 `display2DGrid.m` 中，无返回值）以供参考。输入变量 `grid` 是一个二维矩阵，对应单层平面网格，元胞数组 `nets` 表示所有需布线的线网（包含所有线网的端口）。`display2DGrid(grid, nets)` 函数会依据 `grid` 画出一个平面二维网格，每个格子的颜色由 `grid` 中对应元素的值决定。推荐的函数调用方式是在 `grid` 中，将同一线网布线路径上的所有单元格对应位置处的值都设为相同值（但不同线网采用不同的值），这样最后画出

的每个线网各自的布线路径颜色统一，而不同线网的布线路径颜色不同。
`display2DGrid(grid, nets)`还对线网端口的可视化做了额外处理。如果采用推荐方式调用该函数，每个端口的颜色会与对应线网路径的颜色有轻微的区别。